Indexing All (r, s)-Nucleus Decompositions

Anonymous Author(s)

Abstract

Dense subgraphs carry much of the meaning of a graph, and decomposing a graph into hierarchies of dense subgraphs is a fundamental task in graph mining. The (r, s)-nucleus decomposition is the clique-level member of this family, scoring every r-clique by its surrounding s-cliques. It subsumes both the k-core and the k-truss. The parameter pair (r, s) acts as a resolution, so analysts need many cells of the parameter plane rather than one. Existing algorithms face three obstacles there: astronomically many scored cliques, support counting over even more witness cliques, and a plane of cells each as hard as the last. We observe that vertices belonging to exactly the same maximal cliques are interchangeable, and that the whole plane respects this symmetry. Based on this observation, we propose NSI, an index built on the clique quotient of the graph. Scored cliques collapse into patterns, support counts collapse into polynomial coefficients, and two transfer theorems reduce the whole plane to a single peeled cell. Wherever the closed form is certified, the index stores nothing and reconstructs the answer at query time. On real graphs the index is up to seven orders of magnitude smaller than the explicit table it replaces, and it answers point queries in 74 nanoseconds. Its builder outperforms the state-of-the-art single-cell algorithm by up to 742× in time and 226× in memory. It also completes every cell of an evaluation grid on which the state of the art completes a minority.

1 Introduction

A graph represents entities as vertices and their pairwise relationships as edges. Groups of vertices that are densely connected to one another often carry the meaning of the whole network. For example, in collaboration networks, dense groups correspond to research teams working closely together [10]; in web graphs, they mark clusters of pages on one topic [9]; in biological networks, they capture protein complexes [23]; in financial and social platforms, they expose coordinated fraud rings [2, 13].

Decomposing a graph into a hierarchy of nested dense subgraphs is a fundamental task in graph mining. Representative applications include community detection [10, 24], densest-subgraph discovery [7], influence analysis [15, 18], and graph visualization [21]. In the literature, the task has been studied at three levels of granularity. The k-core peels vertices by degree [3, 31]; the k-truss peels edges by triangle count [35, 38]; the (r, s)-nucleus decomposition peels r-cliques by s-clique count [29]. The nucleus model subsumes the other two: the $(1, 2)$-nucleus is exactly the k-core, and the $(2, 3)$-nucleus is exactly the k-truss [29]. The parameter pair (r, s) therefore acts as a resolution, and every cell of the parameter plane reveals the graph's dense structure at a different granularity [29, 30]. The state-of-the-art algorithm, CND [36], computes the decomposition of one cell at a time. In this paper, we aim to answer every cell of a bounded parameter plane from one index.

For integers $r < s$, consider subgraphs in which every r-clique is contained in at least k s-cliques of the same subgraph. The nucleus value of an r-clique is the largest k for which such a subgraph contains it. Computing the cell (r, s) means computing this value for every r-clique of the graph.

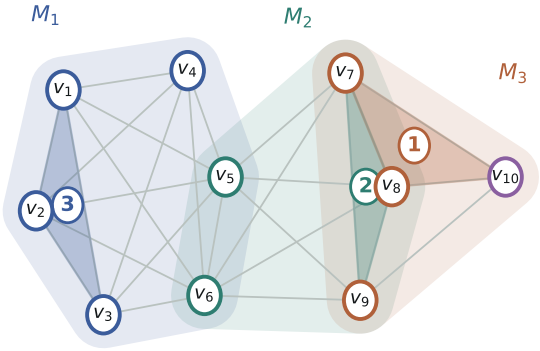

Figure 1: An example graph G and three of its $(3, 4)$-nucleus values.

Example 1.1. Consider the graph G in Figure 1, whose ten vertices form three maximal cliques: M_1 with six vertices, M_2 with five, and M_3 with four. At the cell $(3, 4)$, the triangle $\{v_1, v_2, v_3\}$ has nucleus value 3: it lies in the subgraph induced by M_1, in which every triangle is contained in at least three 4-cliques. The triangle $\{v_7, v_8, v_9\}$ has value 2, and the triangle $\{v_7, v_8, v_{10}\}$ has value 1. Other cells give other resolutions: at $(3, 5)$ the value of $\{v_1, v_2, v_3\}$ is still 3, while at $(4, 5)$ the 4-clique $\{v_1, v_2, v_3, v_4\}$ has value 2.

Applications. Many uses of the decomposition need several cells of the plane rather than one. *Multi-resolution exploration.* An analyst inspecting a dense region rarely knows the right resolution in advance. Low cells surface broad communities, while high cells isolate small, strongly coordinated groups [13, 29]. Exploration therefore issues a stream of queries across cells, and each query must return in interactive time. *Parameter selection.* Which cell best separates signal from noise differs across graphs and tasks [30]. Practitioners choose by probing: compute several cells, compare the resulting hierarchies, and keep the most informative one. Today every probe costs a full decomposition run. *Point queries at query time.* Downstream systems ask point questions: given one concrete group of vertices, how dense is its surrounding structure at each resolution [26]? Answering from the raw graph requires a fresh decomposition per question.

Existing Methods and Key Limitation. The state-of-the-art algorithm CND [36] enumerates every r-clique of the requested cell, counts its supporting s-cliques, and repeatedly removes the clique of minimum count. Careful counting structures let it obtain these counts without listing individual s-cliques, which made single-cell decomposition practical on graphs with millions of edges [36]. However, all existing algorithms share a fundamental limitation: they materialize ***every r-clique of every cell they compute***. The number of r-cliques grows combinatorially with r: web-it-2004, a web graph

with 7.2 million edges, contains more than 1.5×10^{12} 5-cliques. Even a single shallow cell is already prohibitive: on web-it-2004, CND needs 5,581 seconds and 101 GB of memory for the single cell $(3, 4)$. Within a 1,800-second, 300 GB budget, CND finishes none of the nine web-it-2004 cells of our evaluation grid (Section 8).

This leads to the central research question of this paper: ***Can one index answer the nucleus value of every r-clique, at every cell of a bounded plane, without ever materializing the plane?***

Challenges. Answering the plane from one structure faces three obstacles. *Astronomically many objects.* For one small cell, per-clique computation is affordable. However, across a plane the objects multiply twice: the number of r-cliques explodes with r, and every cell owns its own copy of the work. Even the ten-vertex graph of Figure 1 carries 61 cliques of size three to five; on web-it-2004 the count exceeds 10^{12}. Therefore, any approach that touches each r-clique of each cell even once is ruled out from the start. *Counting without enumeration.* Peeling is driven by counts: each removal must decrement the count of every r-clique that shared a supporting s-clique with the removed one. However, s-cliques outnumber r-cliques, and at high s they cannot even be listed once. Therefore, both the initial counts and every later decrement must be obtained without enumerating a single s-clique. *A plane of cells.* A bounded plane contains dozens of cells, and each cell is a full decomposition in its own right. However, running even the best single-cell algorithm once per cell multiplies its cost by the number of cells. Worse, the deep cells are exactly the ones no existing algorithm reaches. Therefore, the computation done in one cell must be made to pay for all the others.

Our Idea. Our solution is built on one structural observation: vertices that belong to exactly the same maximal cliques are interchangeable, and the whole plane respects this symmetry. We call the resulting grouping the *clique quotient* of the graph. *Peel patterns, not cliques.* Under the quotient, an r-clique is described by how many vertices it takes from each group, not by which vertices it takes. The 33 triangles of Figure 1 collapse into 7 such patterns; on real graphs the collapse reaches six orders of magnitude. The peel runs unchanged on patterns, with every step carrying a multiplicity. *Count as a coefficient.* The number of s-cliques containing a pattern is a coefficient of one small polynomial derived from the quotient. Both the initial counts and all later decrements reduce to coefficient extraction, so no s-clique is ever listed. *Certify instead of recompute.* Two transfer theorems link neighboring cells. Along s, a pattern whose value reaches its closed form keeps that form in every later cell. Along r, the starting values of a cell follow from the cell below it. Together they leave a single cell to peel; every other cell of the plane is arithmetic.

The consequence is an index that stores almost nothing. Wherever the closed form is certified, the value is reconstructed at query time from the quotient itself, so certified patterns cost zero bytes. We call the structure NSI, for Nucleus Spectrum Index.

Contributions. Our key contributions are summarized as follows. *The first index over the nucleus plane.* We propose NSI (Section 7), the first index that answers every cell of a bounded (r, s) plane from one structure. On web-it-2004, the explicit table behind the same queries would occupy 50.8 TB, and it cannot be built at all for $r \geq 4$. NSI occupies 1.70 MB and answers a point query in 74 nanoseconds. *A quotient framework with certified transfer.* We develop the clique quotient and its peel (Sections 4–6): interchangeability, coefficient counting, and two transfer certificates that reduce the whole plane to one peeled cell. Disabling the certificates alone slows the build by 22.6× on nasasrb, with answers unchanged. *An applicability test computable in advance.* We characterize exactly when the quotient compresses, by a one-number test computable before any index is built (Section 7). The test predicted every graph of our evaluation, and it also predicted the social networks the evaluation excludes. On soc-pokec the quotient compresses by a factor of only 1.001, and the test reports this without building anything. *Comprehensive Experimental Evaluation.* We evaluate on eight graphs from four domains, all with at least a million edges (Section 8). Against CND run serially on the same machine, our builder wins all 42 grid cells that CND finishes, by up to 5,806×, and completes the other 30 within seconds. On a matched standalone cell it is 742× faster and 226× leaner.

Table 1: Notation used across sections.

Symbol	Meaning
$G = (V, E)$	input graph, $n = \|V\|$, $m = \|E\|$
r, s	scored-clique size and witness-clique size, $r < s$
R, S	an r-clique (scored) and an s-clique (witness)
$\deg_{(s)}(R)$	s-clique support of R (Definition 2.2)
$\kappa_s(R)$	core number of R at witness size s (Definition 2.4)
$\mathcal{M}(G)$	the maximal cliques of G
$\omega(R)$	local clique number: size of the largest clique containing R
$\mathcal{P}$	the indexed plane: $r \in [r_{\min}, r_{\max}]$, $r < s \leq s_{\max}$

2 Preliminaries

In this section, we fix notation, formalize the core number of an r-clique, and state the plane-indexing problem.

Graph notation. We consider a simple, undirected, unweighted graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges. For $V' \subseteq V$, we write $G[V']$ for the subgraph of G induced by V'. Table 1 summarizes the notation used across sections; symbols local to one lemma or algorithm are introduced where they are used.

Definition 2.1 (ℓ-Clique). For an integer $\ell > 0$, a vertex set $X \subseteq V$ with $|X| = \ell$ is an *ℓ-clique* of G if every two distinct vertices of X are adjacent.

Following the convention of nucleus decomposition [29, 36], we call an r-clique a *scored clique*: the decomposition computes a core number for it. An s-clique is a *witness clique*: it certifies the cohesion of the scored cliques it contains.

Maximal cliques. A clique is *maximal* if no outside vertex can be added to it while preserving a clique; $\mathcal{M}(G)$ denotes the set of maximal cliques of G. Every clique of G is contained in at least one maximal clique. We write $\omega(R)$ for the size of the largest clique of G that contains the clique R, and call it the *local clique number* of R. In Figure 1, $\omega(\{v_1, v_2, v_3\}) = 6$ by M_1, $\omega(\{v_7, v_8, v_9\}) = 5$ by M_2, and $\omega(\{v_7, v_8, v_{10}\}) = 4$ by M_3.

Definition 2.2 (s-Clique Support). The *s-clique support* of an r-clique R in G is $\deg_{(s)}(R) = |\{S \subseteq V : R \subseteq S,\ S \text{ is an } s\text{-clique of } G\}|$.

In Figure 1, $\deg_{(4)}(\{v_1, v_2, v_3\}) = 3$: the three 4-cliques obtained by adding v_4, v_5, or v_6.

Nucleus and core number. Cohesion is certified by a family of witness cliques that sustains all of its own scored cliques [29]. For a family $\mathcal{S}$ of s-cliques, write $R(\mathcal{S})$ for the set of r-cliques contained in at least one member of $\mathcal{S}$. Write $G[\mathcal{S}]$ for the subgraph induced by the union of the members.

Definition 2.3 (k-(r,s)-Nucleus). A k-(r,s)-*nucleus* of G is a subgraph $G[\mathcal{S}]$ induced by a family $\mathcal{S}$ of s-cliques such that (i) every $R \in R(\mathcal{S})$ is contained in at least k members of $\mathcal{S}$; (ii) any two members are linked inside $\mathcal{S}$ by a chain of members in which consecutive cliques share at least r vertices; and (iii) $\mathcal{S}$ is maximal under inclusion among families satisfying (i) and (ii).

Support in condition (i) is counted within the family, not in G: a nucleus must sustain its own scored cliques. Condition (ii) links consecutive witnesses through shared scored cliques, since two s-cliques share r vertices exactly when they contain a common r-clique.

Definition 2.4 (Core Number). The *core number* $\kappa_s(R)$ of an r-clique R is the largest k such that $R \in R(\mathcal{S})$ for the defining family $\mathcal{S}$ of some k-(r,s)-nucleus, and 0 if R lies in no s-clique.

The size of the scored clique is implicit in its argument, so the single notation $\kappa_s(R)$ serves every cell of the plane. The nucleus model subsumes the classical models: $(r,s) = (1,2)$ yields the k-core [31] and $(r,s) = (2,3)$ yields the k-truss [29, 38]. In Figure 1, $\kappa_4(\{v_1, v_2, v_3\}) = 3$, as computed in Example 1.1.

Problem statement (plane indexing). A *plane* $\mathcal{P}$ is the set of cells (r,s) with $r_{\min} \le r \le r_{\max}$ and $r < s \le s_{\max}$. Given G and $\mathcal{P}$, the problem is to build an index that, for any s and any r-clique R with $(|R|, s) \in \mathcal{P}$, returns $\kappa_s(R)$. A scored clique contained in no witness has core number 0. The algorithmic sections therefore focus on scored cliques that lie in at least one maximal clique of size at least s.

The next section recalls how the state of the art computes one cell, and measures where its cost concentrates.

3 The State-of-the-Art Approach

In this section, we recall the peeling skeleton that all existing algorithms instantiate, and we measure where its cost concentrates.

The peeling skeleton. Every existing algorithm computes one cell (r,s) by the same skeleton, shown in Algorithm 1 [29, 36]. It materializes the scored cliques, initializes their supports, and repeatedly removes a clique of minimum support. The level k never decreases, and each clique receives the level at which it is removed.

Algorithm 1 enumerates the scored cliques and allocates one mutable record per clique (Line 1). It initializes each support (Line 3) and then peels: extract a minimum (Line 6), assign the current level (Line 7), and credit every scored clique that loses a witness (Lines 8–10). The state of the art, CND [36], realizes Lines 3 and 8–10 by aggregated counting, so no witness clique is ever listed one by one. This counting is what made single cells with millions of edges practical, and we adopt it as our baseline throughout.

Cost anatomy. The skeleton's remaining cost sits in Line 1: every scored clique of the cell exists as a record, and every record carries mutable peel state. Memory is therefore at least proportional to the number of scored cliques, and time to the stream of per-clique updates. Both grow combinatorially with r, as Table 2 shows for web-it-2004.

Algorithm 1: The explicit peel of existing algorithms.

```
   Input: Graph G, cell (r, s)
   Output: κ_s(R) for every r-clique R of G
1  R ← all r-cliques of G;                          // one record per clique
2  foreach R ∈ R do
3      sup(R) ← deg_(s)(R)
4  k ← 0;
5  while R ≠ ∅ do
6      R* ← a clique of minimum sup in R;
7      k ← max(k, sup(R*));   κ_s(R*) ← k;
8      foreach witness S ⊇ R* not yet destroyed do
9          foreach R' ⊂ S with R' ∈ R, R' ≠ R* do
10             sup(R') ← sup(R') − 1
11     remove R* from R;
12 return κ_s
```

Table 2: Scored cliques on web-it-2004 (7.2 million edges).

r	#r-cliques	one record each
3	338,786,461	2.7 GB
4	28,530,662,583	228 GB
5	2,140,698,950,272	17 TB

The second column of Table 2 counts the objects of Line 1; the third charges a conservative eight bytes per record. At $r = 4$ the records alone exceed the memory of a large server, and at $r = 5$ they exceed it by two orders of magnitude. The measured behavior matches: on web-it-2004, CND spends 5,581 seconds and 101 GB on the single shallow cell $(3,4)$. Within a 1,800-second, 300 GB budget, it finishes none of the nine cells of our evaluation grid (Section 8).

What a plane demands. The skeleton is also strictly per cell: nothing computed at (r,s) is reused at $(r,s{+}1)$ or $(r{+}1,s)$. A plane therefore multiplies the cost by its number of cells. Aggregated counting removed the witness side of the cost; the scored side, one record per clique per cell, remains. Removing it requires a representation in which many scored cliques share one record and all cells share one representation. Building that representation is the subject of the next section.

4 The Clique Quotient

Algorithm 1 pays once per scored clique, so its cost is fixed by how many scored cliques exist. This section asks when two scored cliques are genuinely different, and shows that on real graphs most of them are not. We first group vertices, then group cliques, and finally quantify the collapse.

Classes. For a vertex v, let $\mathcal{M}(v) = \{M \in \mathcal{M}(G) : v \in M\}$ denote the maximal cliques that contain it.

Definition 4.1 (Class). A *class* of G is a maximal set of vertices that all have the same maximal-clique membership: u and v belong to one class if and only if $\mathcal{M}(u) = \mathcal{M}(v)$.

Classes partition V, and the number of classes never exceeds n. In Figure 1, the classes are $A = \{v_1, \ldots, v_4\}$ inside M_1 only; $B = \{v_5, v_6\}$ shared by M_1 and M_2; $C = \{v_7, v_8, v_9\}$ shared by M_2 and M_3; and $D = \{v_{10}\}$ inside M_3 only. The vertex colors of the figure encode

exactly this partition. Classes sit cleanly inside maximal cliques, which the following lemma records.

Lemma 4.2 (Class Containment). *Let c be a class and M a maximal clique. Then either every vertex of c belongs to M, or none does.*

Proof. Suppose $u \in c$ lies in M, and let $v \in c$ be any other vertex of the class. Then $M \in \mathcal{M}(u) = \mathcal{M}(v)$, so $v \in M$. □

Interchangeability. Vertices of one class are not merely similar; they are indistinguishable by the graph itself.

Theorem 4.3 (Interchangeability). *Let $u \neq v$ be vertices of one class. Then the transposition that swaps u and v and fixes every other vertex is an automorphism of G. Consequently, for every s and every r-clique R with $u \in R$ and $v \notin R$, $\kappa_s(R) = \kappa_s\big(R \setminus \{u\} \cup \{v\}\big)$.*

Proof. Let M be any maximal clique containing u; since $\mathcal{M}(u) = \mathcal{M}(v)$, it also contains v, so u and v are adjacent. Let w be any neighbor of u other than v. The edge $\{u, w\}$ lies in some maximal clique M', and $M' \in \mathcal{M}(u) = \mathcal{M}(v)$ places v in M', so w is also a neighbor of v. Hence $N[u] \subseteq N[v]$, and by symmetry $N[u] = N[v]$. A transposition of two adjacent vertices with equal closed neighborhoods preserves adjacency, so it is an automorphism. Core numbers depend only on the isomorphism type of G, so every automorphism preserves κ_s. Applying the transposition to R yields $R \setminus \{u\} \cup \{v\}$, which proves the claim. □

In Figure 1, swapping v_1 and v_2 maps the graph onto itself, so every core number that mentions v_1 has a twin that mentions v_2.

Patterns. Interchangeability licenses forgetting which vertices a scored clique uses.

Definition 4.4 (Pattern). The *pattern* of an r-clique R is the multiset of classes of its vertices, written $P = \{c_1^{b_1}, \ldots, c_t^{b_t}\}$ with $b_1 + \cdots + b_t = r$, where R takes b_i vertices from class c_i.

Corollary 4.5. *Two r-cliques with the same pattern have the same core number at every s.*

Proof. Permuting vertices within one class is a product of same-class transpositions, each an automorphism by Theorem 4.3, so it is an automorphism. For two cliques with one pattern, some within-class permutation maps the first onto the second. Automorphisms preserve κ_s, which proves the claim. □

Core numbers are therefore functions of patterns, and we write $\kappa_s(P)$ for the shared value. The peel of Algorithm 1 can run on patterns directly, provided each pattern knows how many cliques it stands for.

Lemma 4.6 (Pattern Multiplicity). *Let $P = \{c_1^{b_1}, \ldots, c_t^{b_t}\}$ be the pattern of at least one r-clique, and let $n_i = |c_i|$. Then the number of r-cliques with pattern P is exactly $\prod_{i=1}^{t} \binom{n_i}{b_i}$.*

Proof. Some r-clique with pattern P lies in a maximal clique M, so each c_i intersects M, and Lemma 4.2 puts each c_i wholly inside M. All vertices of $c_1 \cup \cdots \cup c_t$ are then pairwise adjacent, so every choice of b_i vertices from each c_i forms an r-clique, and each such clique has pattern P. The number of choices is $\prod_i \binom{n_i}{b_i}$. □

Table 3: The patterns of G at $r = 3$.

pattern	multiplicity	#cliques
$\{A^3\}$	$\binom{4}{3}$	4
$\{A^2B\}$	$\binom{4}{2}\binom{2}{1}$	12
$\{AB^2\}$	$\binom{4}{1}\binom{2}{2}$	4
$\{B^2C\}$	$\binom{2}{2}\binom{3}{1}$	3
$\{BC^2\}$	$\binom{2}{1}\binom{3}{2}$	6
$\{C^3\}$	$\binom{3}{3}$	1
$\{C^2D\}$	$\binom{3}{2}\binom{1}{1}$	3
7 patterns		33

The collapse, quantified. Summing Lemma 4.6 over the realized patterns counts all scored cliques. The ratio of cliques to patterns is therefore the average multiplicity:

$$\frac{\#r\text{-cliques}}{\#\text{patterns}} = \operatorname*{avg}_{P} \prod_{i} \binom{n_i}{b_i}. \tag{1}$$

The ratio is driven by class sizes: one class of hundreds of vertices contributes binomials that dwarf any number of singleton classes.

Example 4.7. Consider the graph G in Figure 1 at $r = 3$. Its 33 triangles realize the 7 patterns of Table 3; for instance, all 12 triangles with pattern $\{A^2B\}$ share the core number computed once in Example 1.1. Across the whole plane $r \in [3, 5]$, $s \leq 8$, the 61 scored cliques of G realize 16 patterns.

On real graphs the collapse is far larger. On pkustk13, a finite-element mesh, 94,893 vertices form 31,872 classes, and the 2.1×10^{12} 5-cliques of web-it-2004 realize 1.3 million patterns, a collapse of six orders of magnitude. Section 7 returns to Equation (1) as a predictor: the average is computable before any decomposition, and it decides in advance whether the quotient is worth building.

Patterns are now the unit of peeling. The peel still needs their supports, and obtaining those without enumerating witnesses is the subject of the next section.

5 Peeling in the Quotient

Patterns are the peel unit, but the peel is driven by supports, and supports count witnesses. This section computes every support, and every update to it, without listing a single witness clique. We first partition the witnesses into boxes, then express support as one coefficient, and then assemble the cell algorithm PeelCell.

Boxes. Inside one maximal clique, counting is trivial. All classes of M_1 in Figure 1 lie wholly inside it (Lemma 4.2), so the witnesses inside M_1 are counted by binomials over class sizes. Across maximal cliques, however, witnesses of overlaps would be counted twice. The remedy is a classical pivoting argument [12, 36], lifted from vertices to classes: the witnesses split into disjoint groups, each with a product structure.

Definition 5.1 (Box). A *box* B is a set of classes C_B, together with per-class bounds $\ell_c \leq u_c \leq n_c$ for $c \in C_B$, where $n_c = |c|$. A clique *belongs to* B if it uses y_c vertices of each class $c \in C_B$ with $\ell_c \leq y_c \leq u_c$, and no vertex outside C_B.

Lemma 5.2 (Box Partition). *From $\mathcal{M}(G)$ one can compute, once for the whole plane, a family $\mathcal{B}$ of boxes such that every clique of*

every size $s \le s_{\max}$ belongs to exactly one box, and the classes of each box are pairwise adjacent.

The construction runs a pivoted recursion over the class graph and is deferred to Appendix A.1; only the two properties above are used in the sequel.

Example 5.3. Consider the graph G in Figure 1 with $s_{\max} = 8$. Three boxes suffice: $B_1 = \{A, B\}$ with free bounds, $B_2 = \{B, C\}$ requiring at least one C vertex, and $B_3 = \{C, D\}$ requiring at least one D vertex. The two lower bounds make the split disjoint at every size. At $s = 4$ the boxes hold $\binom{6}{4} = 15$, $\binom{5}{4} = 5$, and $\binom{4}{4} = 1$ witnesses: 21 in total, and each witness in exactly one box. At $s = 5$ the counts are 6, 1, and 0, matching the 7 witnesses of G.

Support as one coefficient. Fix a box B and a pattern P whose classes all lie in C_B, with composition b on those classes. During the peel, some patterns have already been removed; on box B, let A_B collect the compositions of the removed patterns hosted by B.

Definition 5.4 (Box Coefficient). For a box B, a composition b, a witness size s, and a set A_B of removed compositions,

$$\mathrm{Coef}_s(B, b, A_B) \;=\; \sum_{\substack{y:\ \ell \vee b \le y \le u,\ \sum_c y_c = s \\ \nexists\, a \in A_B:\ a \le y}} \prod_{c \in C_B} \binom{n_c - b_c}{y_c - b_c}. \qquad (2)$$

With $A_B = \emptyset$, Equation (2) is the coefficient of x^s in the polynomial $\prod_{c \in C_B} \left(\sum_j \binom{n_c - b_c}{j - b_c} x^j\right)$, and is computed in $O(|C_B| \cdot s)$ time by one pass of dynamic programming. A witness is *surviving* if it contains no removed scored clique.

Lemma 5.5 (Residual Support). *At any point of a peel that removes whole patterns, and for any instance R of a pattern P, the number of surviving witnesses of size s containing R equals $\sum_B \mathrm{Coef}_s(B, b_P, A_B)$, summed over the boxes hosting P.*

Proof. By Lemma 5.2, the witnesses containing R split disjointly over the boxes whose classes cover P, so it suffices to count within one box B. The witnesses of B containing R are the choices of $y_c - b_c$ further vertices per class outside the b_c vertices of R, giving the product of Equation (2) for each admissible y. It remains to show a witness W with composition y is destroyed exactly when $a \le y$ for some removed composition $a \in A_B$. If $a \le y$, then choosing a_c vertices of W per class yields a removed scored clique inside W, since the classes of B are pairwise adjacent. Conversely, a removed clique inside W uses at most y_c vertices per class, so its composition a satisfies $a \le y$. Filtering these y out of the sum is precisely the third constraint of Equation (2). □

One formula therefore serves the whole peel: initialization is the case $A_B = \emptyset$, and every deletion is one insertion into A_B. The implementation never expands the filtered sum; it credits the compositions that die at each removal, as detailed in Appendix A.3.

The cell algorithm. Algorithm 2 assembles the pieces.

PeelCell initializes every support by the closed form (Line 2), then peels patterns exactly as Algorithm 1 peels cliques: extract a minimum (Line 5), assign the level (Line 6), record the removal in each hosting box (Line 8), and refresh the neighbors' supports through Lemma 5.5 (Line 10). No witness clique appears anywhere: Lines 2 and 10 manipulate coefficients, and Line 8 manipulates compositions.

Algorithm 2: PeelCell: peel one cell in the quotient.

```
   Input: Boxes B, patterns of size r with multiplicities, cell (r, s)
   Output: κ_s(P) for every pattern P
1  foreach pattern P do
2      sup(P) ← Σ_B Coef_s(B, b_P, ∅)
3  k ← 0;   A_B ← ∅ for every box B;
4  while some pattern is unpeeled do
5      P* ← an unpeeled pattern of minimum sup;
6      k ← max(k, sup(P*));   κ_s(P*) ← k;
7      foreach box B hosting P* do
8          insert b_{P*} into A_B
9      foreach unpeeled pattern Q sharing a box with P* do
10         sup(Q) ← Σ_B Coef_s(B, b_Q, A_B)
11     mark P* peeled;
12 return κ_s
```

Theorem 5.6 (Orbit Peel). *Algorithm 2 outputs $\kappa_s(P) = \kappa_s(R)$ for every pattern P and every instance R of P.*

Proof. The removed set is always a union of whole patterns, hence invariant under every within-class automorphism of Corollary 4.5. Such automorphisms map surviving witnesses to surviving witnesses, so at every step all instances of one pattern have equal surviving support, and Lemma 5.5 computes it. The output of the explicit peel does not depend on which minimum-support clique is removed first [29], so we may run Algorithm 1 exhausting one minimum pattern at a time. While an orbit is being exhausted, internal removals only lower the supports of its remaining instances, which stay minimal and receive the same level k by Line 7 of Algorithm 1. Between removals of other patterns, supports evolve exactly as A_B records. Hence Algorithm 2 simulates a valid run of Algorithm 1 and assigns each pattern the common core number of its instances. □

Example 5.7. Consider the cell $(3, 4)$ of the graph G in Figure 1. Line 2 gives $sup(\{A^2B\}) = 3$ on B_1, $sup(\{C^3\}) = 2 + 1 = 3$ across B_2 and B_3, and $sup(\{C^2D\}) = 1$. The first extraction removes $\{C^2D\}$ at level 1, inserting $(2, 1)$ into A_{B_3}. The refresh of $\{C^3\}$ now filters every $y \ge (3, 0)$ with $y \ge (2, 1)$ out of B_3, so its support drops to 2: the removal cascaded through a shared witness, and Equation (2) accounted for it without listing that witness. Two further rounds remove the patterns at levels 2 and 3, reproducing the values of Example 1.1 for all 33 triangles at once.

Complexity. Line 2 costs $O(|C_B| \cdot s)$ per hosted pattern and box. Each removal inserts one composition per hosting box and refreshes only the patterns sharing a box, by crediting the compositions that die (Appendix A.3). The peel therefore scales with patterns and their box incidences, never with cliques. On pkustk13 at $(3, 4)$, PeelCell peels 2.2 million patterns in place of 45.2 million scored cliques. At $r = 5$ the gap widens to 26.8 million against 2.28 billion.

One cell is now affordable. The next section makes one cell pay for the entire plane.

6 From One Cell to the Plane

PeelCell prices one cell, but the plane contains dozens, and each is a full decomposition. This section proves two transfer theorems: values travel along s within a row, and across the boundary from

one row to the next. Together they leave a single cell to peel; every other cell of the plane follows by arithmetic.

The clique floor. The proofs work with a support form of the core number, which the peel view of Section 3 suggests [29].

Lemma 6.1 (Max–Min Form). *For every r-clique R, the core number $\kappa_s(R)$ equals the largest k for which some family $\mathcal{S}$ of s-cliques covers R and every r-clique covered by $\mathcal{S}$ lies in at least k members.*

Proof. The defining family of a nucleus witnessing $\kappa_s(R)$ satisfies the condition, so the maximum is at least $\kappa_s(R)$. Conversely, let $\mathcal{S}$ attain the maximum k, and let $\mathcal{S}_0$ be the members reachable from a clique through R by chains of members sharing r vertices. Two members that contain a common r-clique share r vertices, so every r-clique covered by $\mathcal{S}_0$ has all its containing members inside $\mathcal{S}_0$, and its support is unchanged. Hence $\mathcal{S}_0$ is k-admissible and s-connected, and by finiteness it extends to a maximal such family. That family defines a k-(r,s)-nucleus whose scored cliques include R, so $\kappa_s(R) \geq k$. □

Lemma 6.2 (Clique Floor). *For every r-clique R and every s, $\kappa_s(R) \geq \binom{\omega(R)-r}{s-r}$.*

Proof. If $\omega(R) < s$ the right side is 0 and there is nothing to prove. Otherwise let K be a largest clique containing R and let $\mathcal{S}$ be all s-cliques inside K. Every r-clique covered by $\mathcal{S}$ lies in K and is contained in exactly $\binom{|K|-r}{s-r}$ members, and some member contains R. Lemma 6.1 gives the bound. □

Both ω and the floor are pattern-level quantities, since automorphisms preserve them; we write $\omega(P)$ for the shared value. In Example 1.1, every computed value equals its floor: $6-3$, $5-3$, and $4-3$. The two theorems below explain why this is the typical situation, and turn it into a certificate.

Ascending in s. The transfer along a row rests on a shadow argument in the style of Kruskal and Katona [14, 16]. For a real $x \geq a$, write $\binom{x}{a}$ for $x(x-1)\cdots(x-a+1)/a!$, and let $g_a(m) = \binom{x}{a-1}$ where x solves $m = \binom{x}{a}$; this is the Lovász form of the shadow function [20].

Lemma 6.3 (Shadow Bound). *For $s \geq r+2$ and every r-clique R, $\kappa_{s-1}(R) \geq g_{s-r}\big(\kappa_s(R)\big)$.*

The proof lowers a witness family by one vertex and bounds the loss by the Kruskal–Katona theorem; it appears in Appendix A.2.

Theorem 6.4 (Absorption). *Let R be an r-clique and $s \geq r+2$. If $\kappa_{s-1}(R) = \binom{\omega(R)-r}{s-1-r}$, then $\kappa_s(R) = \binom{\omega(R)-r}{s-r}$.*

Proof. Write $w = \omega(R)$ and $a = s-r$. The floor gives $\kappa_s(R) \geq \binom{w-r}{a}$, and for $w < s$ both sides are 0, so assume $w \geq s$ and suppose $\kappa_s(R) \geq \binom{w-r}{a} + 1$. Lemma 6.3 then gives $\kappa_{s-1}(R) \geq g_a\big(\binom{w-r}{a} + 1\big)$. The solution $x(m)$ of $\binom{x}{a} = m$ increases strictly with m, and $x \mapsto \binom{x}{a-1}$ increases strictly for $x > a-1$. At the integer point $x = w-r \geq a$ this yields $g_a\big(\binom{w-r}{a} + 1\big) > \binom{w-r}{a-1} = \kappa_{s-1}(R)$, a contradiction. □

Corollary 6.5 (Absorbing Chain). *If $\kappa_s(R)$ equals its floor, then $\kappa_{s'}(R)$ equals its floor for every $s' > s$.*

Proof. The conclusion of Theorem 6.4 is the hypothesis at $s+1$ with the same $\omega(R)$; induction on s' finishes the proof. □

Algorithm 3: Build: compute every cell of the plane.

```
Input: Graph G, plane P with r ∈ [r_min, r_max], s ≤ s_max
Output: κ_s(P) for every pattern P and every cell of P
1  compute M(G), the classes, and the boxes B;
2  for r ← r_min to r_max do
3      enumerate the patterns of size r, with multiplicities and ω;
4      if r = r_min then
5          κ_{r+1} ← PeelCell(B, patterns, (r, r+1));
6      else
7          foreach pattern P with min_{c:b_c≥1} κ_r(P−c) − 1 = ω(P) − r do
8              certify κ_{r+1}(P) ← ω(P) − r
9          peel the rest by PeelCell, seeded with the certified values;
10     for s ← r + 2 to s_max do
11         foreach pattern P with κ_{s−1}(P) = C(ω(P)−r, s−1−r) do
12             certify κ_s(P) ← C(ω(P)−r, s−r)
13         peel the rest by PeelCell, seeded with the certified values;
14 return all κ
```

One integer comparison per pattern therefore certifies a whole cell from its left neighbor, and certification, once gained, is never lost. The comparison is also what the plane's cost hinges on. Disabling it and peeling every cell in full slows the whole build by 22.6× on nasasrb, and single cells by 67× to 5,406× (Section 8).

Stepping in r. The second certificate crosses rows at their boundary cells $(r, r+1)$ and $(r+1, r+2)$.

Theorem 6.6 (Diagonal Ceiling). *For every $(r+1)$-clique R^+, $\kappa_{r+2}(R^+) \leq \min_{v \in R^+} \kappa_{r+1}\big(R^+ \setminus \{v\}\big) - 1$.*

Proof. Let $\mathcal{S}$ realize $k = \kappa_{r+2}(R^+)$ in Lemma 6.1, and let $\mathcal{S}'$ be all $(r+1)$-cliques contained in members of $\mathcal{S}$. Let R' be any r-clique covered by $\mathcal{S}'$, and pick a member $W = R' \cup \{u\}$ of $\mathcal{S}'$ containing it. W is covered by $\mathcal{S}$, so distinct members $S_1, \ldots, S_k$ of $\mathcal{S}$ contain W; write $S_i = W \cup \{w_i\}$ with the w_i pairwise distinct and outside W. Each $R' \cup \{w_i\}$ is an $(r+1)$-subset of the clique S_i, hence a member of $\mathcal{S}'$, and these members differ from W and from one another. Therefore R' lies in at least $k+1$ members of $\mathcal{S}'$. Since R^+ lies in a member of $\mathcal{S}$, the family $\mathcal{S}'$ covers every face $R^+ \setminus \{v\}$, and Lemma 6.1 gives $\kappa_{r+1}(R^+ \setminus \{v\}) \geq k+1$ for each. □

Corollary 6.7 (Diagonal Certificate). *If $\min_{v \in R^+} \kappa_{r+1}(R^+ \setminus \{v\}) = \omega(R^+) - r$, then $\kappa_{r+2}(R^+) = \omega(R^+) - r - 1$.*

Proof. Theorem 6.6 bounds $\kappa_{r+2}(R^+)$ from above by $\omega(R^+) - r - 1$, and Lemma 6.2 matches it from below. □

Remark 6.8. The case $r = 1$ of Theorem 6.6 is the classical relation between the trussness of an edge and the coreness of its endpoints [11, 38]; the theorem extends it to every consecutive pair of rows.

At pattern level the faces of P are the patterns $P - c$ with $b_c \geq 1$, so the certificate is again one comparison per pattern. It guards exactly the cells the absorption cannot reach, the first cell of each row. Disabling it slows the build by 2.4× to 3.0× while every other cell stays certified (Section 8).

The plane algorithm. Algorithm 3 assembles the plane sweep.

Build computes the shared front end once (Line 1): the same classes and boxes serve every cell of the plane. The first row is peeled cold at its boundary (Line 5). Every later boundary is filled by the diagonal certificate from the row below (Lines 7–8), and every later cell of a row by absorption from its left neighbor (Lines 11–12).

Patterns that neither certificate reaches are the *residue*; PeelCell peels them with the certified patterns entering the removal order at their known levels (Lines 9 and 13). Seeding is sound by the certification principle: where a lower and an upper bound meet, the value is known before any peeling.

Example 6.9. Consider the graph G of Figure 1 with the plane $r \in [3, 5]$, $s \leq 8$: twelve cells. Build peels exactly one of them, the cell $(3, 4)$ of Example 5.7. Every value of row 3 equals its floor, so absorption certifies the cells $(3, 5)$ through $(3, 8)$ outright. At the boundary $(4, 5)$, the pattern $\{A^4\}$ has faces $\{A^3\}$ with $\kappa_4 = 3$, so the ceiling is 2 and meets the floor $6 - 4$; the same happens for every pattern of rows 4 and 5. Twelve cells, one peel.

On real graphs the residue is just as thin. On pkustk13, the boundary of row 4 leaves 1 uncertified pattern out of 8.66 million, and the boundary of row 5 leaves none out of 26.8 million. The whole plane of a graph with 3.3 million edges is one boundary peel plus arithmetic.

Everything the plane needs is now computed. The next section shows that almost none of it needs to be stored.

7 The Index and Its Query

Build computes the whole plane, and this section decides what of it to keep. The answer is almost nothing: the certified part of the plane reconstructs itself at query time, so the index stores the quotient, the residue, and no more.

What is stored. By Corollary 6.5, once a pattern's value reaches its floor it stays there. Each pattern P of row r therefore has a first such witness size, which we denote $\sigma(P)$. For $s \geq \sigma(P)$ the value is the floor $\binom{\omega(P)-r}{s-r}$; for $s < \sigma(P)$ it is not, and those values are the row's *residue*. Every pattern certifies eventually: for $s > \omega(P)$ no witness inside a clique remains and both the value and the floor are 0, so $\sigma(P) \leq \omega(P) + 1$. NSI therefore stores three blocks: (i) the class label of every vertex; (ii) for every class c, its size and the list $\mathcal{M}(c)$ of maximal cliques containing it; (iii) for every row r, the patterns with $\sigma(P) > r + 1$, each with $\sigma(P)$ and its residue values. Blocks (i) and (ii) are the quotient itself, shared by every row and every cell. Block (iii) is the only per-row data, and on our evaluation graphs it occupies between 0.1 and 29.5 percent of the file; the quotient is the rest.

Reconstruction. The floor needs ω, and the quotient already knows it.

Lemma 7.1 (Reconstruction). *Let $R = \{v_1, \ldots, v_r\}$ be an r-clique with classes $c_1, \ldots, c_r$. Then a maximal clique M contains R if and only if $M \in \bigcap_i \mathcal{M}(c_i)$, and $\omega(R) = \max\{|M| : M \in \bigcap_i \mathcal{M}(c_i)\}$, where each $|M|$ is the sum of $|c|$ over the classes c with $M \in \mathcal{M}(c)$.*

Proof. If M contains R, it contains a vertex of each c_i, and Lemma 4.2 puts every c_i wholly inside M, so $M \in \mathcal{M}(c_i)$ for each i. Conversely, $M \in \bigcap_i \mathcal{M}(c_i)$ places every v_i in M. The largest clique containing R is a maximal clique containing R, which gives the expression for $\omega(R)$. Finally, the classes partition V, and M is the disjoint union of the classes it contains, again by Lemma 4.2, which gives $|M|$. □

Algorithm 4: Query: answer κ_s for given vertices.

Input: Distinct vertices $v_1, \ldots, v_r$ and s with $(r, s) \in \mathcal{P}$
Output: $\kappa_s(\{v_1, \ldots, v_r\})$, or 0 if the vertices form no clique
1 $c_i \leftarrow$ class of v_i for all i; $P \leftarrow$ the multiset $\{c_1, \ldots, c_r\}$;
2 $\Omega \leftarrow \bigcap_i \mathcal{M}(c_i)$;
3 **if** $\Omega = \emptyset$ **then return** 0;
4 **if** P *is stored with* $s < \sigma(P)$ **then return** *the stored value*;
5 **return** $\binom{\max_{M \in \Omega} |M| - r}{s-r}$;

Corollary 7.2 (Zero-Byte Certificates). *A pattern with $\sigma(P) = r+1$ contributes nothing to the index: every one of its values is $\binom{\omega(P)-r}{s-r}$, and $\omega(P)$ is recomputed from blocks (i) and (ii) at query time.*

On pkustk13, all 26.8 million patterns of row 5 and all but one of row 4 are such patterns. This is why the index is small: the plane's answers live in the quotient, not beside it.

Corollary 7.3 (Membership for Free). *For distinct vertices $v_1, \ldots, v_r$, the set $\{v_1, \ldots, v_r\}$ is a clique if and only if $\bigcap_i \mathcal{M}(c_i) \neq \emptyset$.*

Proof. A clique lies in some maximal clique, which then lies in every $\mathcal{M}(c_i)$ by Lemma 7.1. Conversely, a common maximal clique contains all v_i, which are then pairwise adjacent. □

The query. Algorithm 4 answers a point query; the graph itself is never touched.

Theorem 7.4 (Query Correctness). *Algorithm 4 returns $\kappa_s(\{v_1, \ldots, v_r\})$ whenever the input vertices form a clique, and 0 otherwise.*

Proof. If the vertices form no clique, Corollary 7.3 makes Line 2 empty and Line 3 returns 0. Otherwise the input is an r-clique R with pattern P, and $\kappa_s(R) = \kappa_s(P)$ by Corollary 4.5. If $s < \sigma(P)$, the value is a residue value, and Line 4 returns what Build stored. If $s \geq \sigma(P)$, the value equals the floor by Corollary 6.5, and Line 5 computes it with $\omega(R)$ supplied by Lemma 7.1. □

The intersection of Line 2 walks the shortest membership list and probes the others, largest maximal clique first, so the maximum of Line 5 is found at the first common member. A row query, all admissible s at once, resolves P and Ω once and reuses them.

When to build. Equation (1) is computable after the pattern enumeration, before any peeling and before any storage is committed. It is the build decision: a ratio near 1 means the graph has no interchangeable vertices to exploit, and the index degenerates toward the explicit table. On soc-pokec, a social network, the ratio is 1.001, and the test rejects the build without running it. On the mesh nasasrb the ratio is 8.3, on the web graph web-it-2004 it is 515, and the test accepts every graph of Section 8. The applicability of NSI is thus itself a queryable quantity, settled in advance on the cheap side of the computation.

8 Experiments

In this section, we evaluate the builder against the state of the art, and the index against the only baseline that answers the same queries.

Hardware. All experiments run on a server with two Intel Xeon Gold 6342 processors and 503 GB of memory, on a single thread[1]. All algorithms are implemented in C++ and compiled with -O3.

[1] Code will be released upon publication.

Table 4: Datasets: four domains, eight graphs.

	graph	$\|V\|$	$\|E\|$
web	web-it-2004 (WI)	509,338	7,178,413
web	web-Google (WG)	875,713	5,105,039
collaboration	com-dblp (DB)	317,080	1,049,866
co-purchase	com-amazon (AZ)	334,863	925,872
mesh	nasasrb (NS)	54,870	1,311,227
mesh	pkustk11 (PK1)	87,804	2,565,054
mesh	pkustk13 (PK3)	94,893	3,260,967
mesh	pwtk (PW)	217,918	5,708,253

Algorithms. CND is the published state of the art [36], run from its authors' implementation one cell at a time. Build is our plane builder (Algorithm 3); Query is the index query (Algorithm 4). The query baseline is the *archive*: one sorted record per scored clique holding its values across the row, probed by binary search. Every archive choice favors the archive: four-byte keys and values, rows counted at their exact number, and a workload that always hits.

Datasets. Table 4 lists the eight graphs: two web crawls [17, 25], a collaboration network and a co-purchase network from SNAP [17], and four finite-element meshes from the SuiteSparse collection [6]. The indexed plane is $r \in [3, 5]$, $s \le 8$ throughout; the grid experiments use the nine cells $s \le r + 3$ of that plane.

Metrics. Time is end-to-end and memory is the peak of the process, both of a single run on an idle machine. CND receives a 1,800-second budget and a 300 GB cap per cell. A red × marks a cell over the budget, a red − marks a memory kill, and the dotted line in the figures marks the budget itself. Figures label a cell (r, s) compactly as rs. Build computes a whole row per sweep, so a cell is charged its marginal time and the row's peak memory. Times below the harness resolution of 0.01 seconds are drawn at that resolution.

Exp-1: Build time against the state of the art. Figure 2 shows build time per grid cell on all eight graphs. The y-axis is time in seconds on a log scale, and the x-axis lists the cells of each row. CND completes 42 of the 72 cells; Build completes all 72. It is faster on every one of the 42. The closest cell is WG at $(3, 4)$ with 1.4×; the widest is DB at $(5, 8)$ with 5,806×. The gap widens along both parameters, exactly as the certificates predict. Within a row, absorption turns later cells into arithmetic: on DB, 43.31 seconds against 0.08 at $(4, 6)$. Across rows, the diagonal starts each boundary warm. On WI, CND finishes none of the nine cells, while our whole rows take 0.81, 1.60, and 3.18 seconds. On a single matched cell computed standalone, WI at $(3, 4)$, CND needs 5,581 seconds and 101 GB against our 7.5 seconds and 0.45 GB. That is 742× in time and 226× in memory.

Exp-2: Build memory against the state of the art. Figure 3 shows peak memory for the same cells. CND's memory follows its scored-clique count and crosses 93 GB already on the one-million-edge DB at row 5. Two mesh cells and one WI cell die at the 300 GB cap. Our peak follows the pattern count instead and stays between 36 MB and 57 GB across all 72 cells. The one graph where CND is leaner at $r = 3$, WG, is the graph with the weakest quotient of the eight; its rows still win on time.

Exp-3: Building the whole plane. Table 5 reports the end-to-end cost of Build for the full plane $r \in [3, 5]$, $s \le 8$. Every graph builds in minutes, six of eight in under two, on one thread. For scale, the one standalone CND cell of Exp-1 costs 207× the whole WI plane.

Table 5: Whole-plane build: time and peak memory.

graph	time (s)	memory (GB)
WI	27.0	1.7
WG	599.1	25.8
DB	66.5	5.2
AZ	3.6	0.6
NS	87.8	8.8
PK1	12.7	1.7
PK3	217.1	25.9
PW	70.6	7.3

Table 6: Size of the archive and of NSI for the whole plane.

graph	archive	NSI	ratio
WI	50.8 TB	1.70 MB	3.0×10^7
PK3	86.4 GB	0.51 MB	1.7×10^5
PW	58.5 GB	1.21 MB	4.8×10^4
PK1	28.2 GB	0.32 MB	8.8×10^4
NS	9.9 GB	0.51 MB	1.9×10^4
DB	8.5 GB	1.65 MB	5.1×10^3
WG	4.9 GB	8.81 MB	5.5×10^2
AZ	21.7 MB	1.53 MB	14

Table 7: Warm point-query latency (ns) and speedups over the archive probe.

	$r = 3$			$r = 4$			$r = 5$		
graph	NSI	arch.	speedup	NSI	arch.	speedup	NSI	arch.	speedup
WI	79	942	11.9×	74	–	–	80	–	–
WG	354	632	1.8×	401	820	2.0×	414	1016	2.5×
DB	173	397	2.3×	132	597	4.5×	156	1023	6.6×
AZ	376	490	1.3×	433	388	0.9×	408	226	0.6×
NS	199	632	3.2×	354	879	2.5×	401	1160	2.9×
PK1	163	708	4.3×	292	1031	3.5×	344	1372	4.0×
PK3	282	798	2.8×	221	1207	5.5×	426	1582	3.7×
PW	177	829	4.7×	185	1143	6.2×	204	2598	12.7×

Exp-4: Index size against the archive. Table 6 compares NSI against the archive over the same plane. The archive column counts its exact rows at four bytes per key and cell; the NSI column is the file as written. On WI the archive would occupy 50.8 TB across 1.59 trillion rows, and for $r \ge 4$ it cannot be materialized at all; NSI answers the same queries from 1.70 MB. The ratio tracks the quotient: it is seven orders of magnitude on WI and smallest on AZ, the graph whose archive is itself only 20.7 MB.

Exp-5: Query latency against the archive. Table 7 reports the median point-query latency of Query against the archive probe, on 20,000 uniformly sampled scored cliques per row. Warm queries repeat over a resident working set; cold queries follow a cache eviction. Query answers in 74 to 433 nanoseconds warm, and it wins 20 of the 22 comparisons the archive can enter, by up to 12.7×. On WI at $r \ge 4$ the archive cannot be built, so the index is the only structure that answers at all. The archive wins only on AZ at $r \ge 4$: its 22 MB fit in cache, and a graph whose archive fits in cache does not need an index.

Exp-6: The certificates, ablated. Table 8 disables each transfer theorem in turn and rebuilds the plane. Without absorption, every

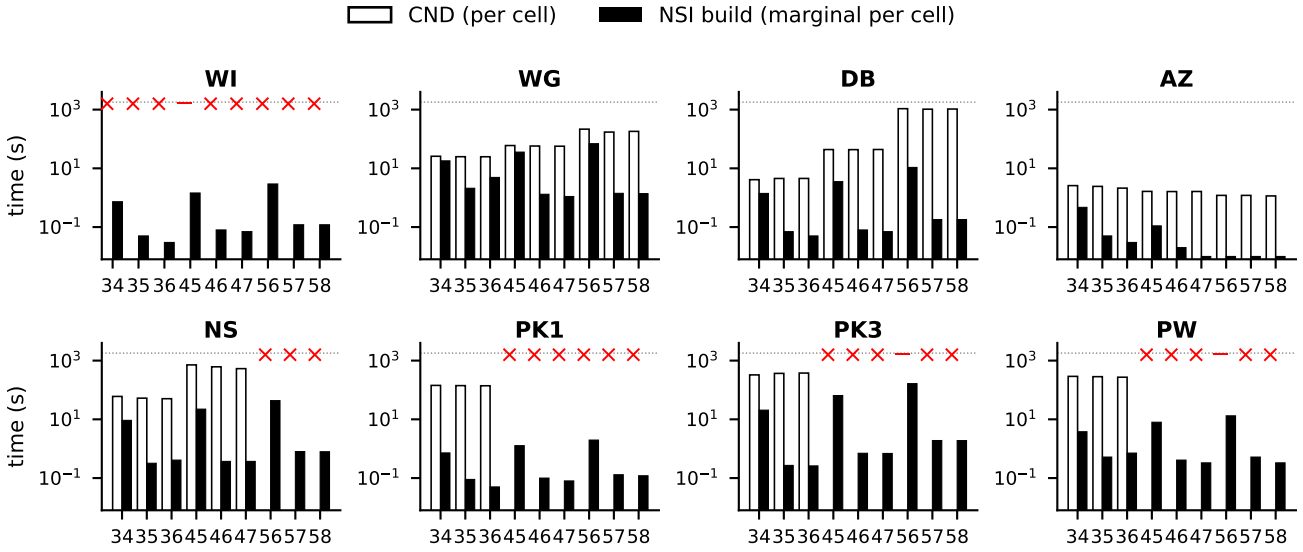

Figure 2: Build time per grid cell (log scale).

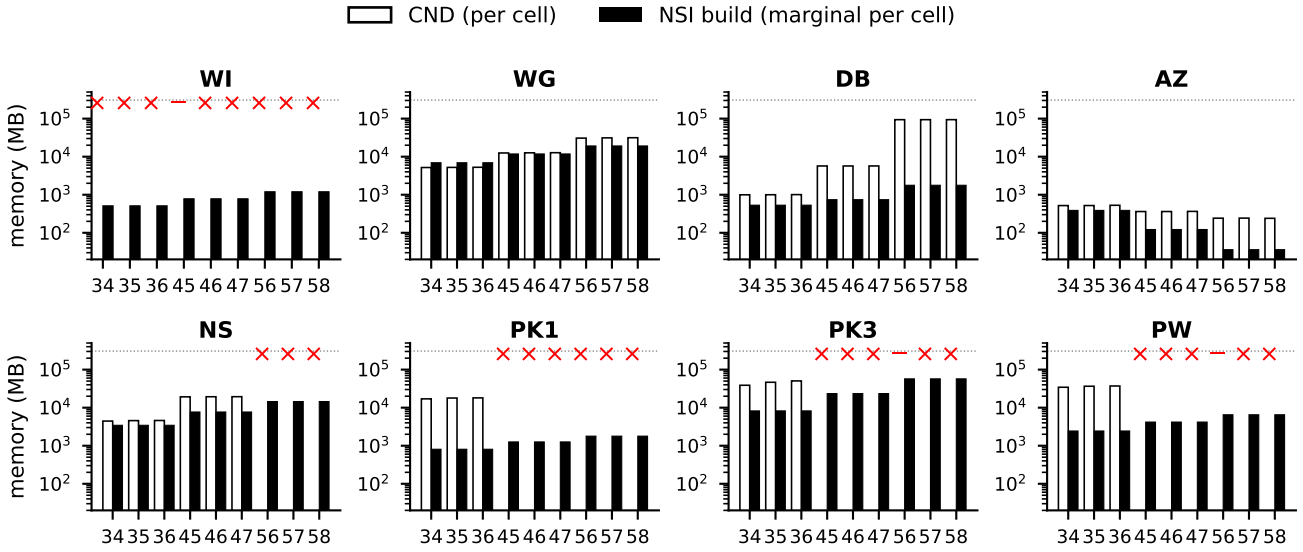

Figure 3: Build memory per grid cell (log scale).

Table 8: Whole-plane build time under ablations.

graph	default	no diagonal	no absorption
NS	83.7 s	197.9 s (2.4×)	1,894.6 s (22.6×)
PK3	204.5 s	615.8 s (3.0×)	> 5,400 s (> 26×)

cell above a boundary is peeled in full: NS slows by 22.6×, and PK3 exceeds a 5,400-second budget against its 204-second default. Without the diagonal, every boundary is peeled cold: 2.4× on NS and 3.0× on PK3, with memory more than doubling. The completed ablations return the same core-number distribution as the default build.

Exp-7: Applicability, decided in advance. The build test of Section 7 was computed for every candidate graph before any build. It accepted all eight graphs of Table 4, and it rejected the social networks: soc-pokec at a ratio of 1.001 and email-Eu-core at 1.002. In those graphs the vertices are pairwise distinguishable, so the quotient is the graph itself. The rejections cost the front end only, and no index was built to learn them.

9 Related Work

Cohesive decompositions. The k-core [3, 31] and the k-truss [5, 11, 35, 38] are the classical peel-based decompositions, and the (r, s)-nucleus generalizes both to cliques [29]. The interplay between the

core and truss levels has been studied empirically [19]; Theorem 6.6 gives the formal inequality, for every consecutive pair of rows. The work [32] accelerates nucleus decomposition and its hierarchy, [27, 33] construct the nucleus hierarchy efficiently, and [28] computes nuclei locally; all of them, like CND [36], compute one cell at a time over explicit scored cliques, whereas we answer a bounded plane of cells from one structure. Parallel and distributed core computation [22**?**] scales single cells rather than removing their per-object cost.

Clique counting and indexes. Pivot-based clique counting [4, 12, 34] counts cliques without listing them, and CND brought aggregated counting into nucleus decomposition [36]; our boxes lift the same pivoting from vertices to classes, so counting serves patterns instead of single cliques. Index structures for cohesive search, such as the truss-equivalence index [1] and core-maintenance structures [37], support one fixed decomposition; NSI instead stores the quotient from which a whole plane of decompositions is reconstructed. Treating vertices with identical neighborhoods as one object is classical in graph compression; our classes coarsen by maximal-clique membership, which is exactly the granularity the nucleus values respect.

10 Conclusion

In this paper, we presented NSI, the first index that answers every cell of a bounded (r, s)-nucleus plane from one structure. The clique quotient collapses scored cliques into patterns, coefficients replace witness enumeration, and two transfer certificates reduce the whole plane to a single peeled cell. The certified part of the plane is reconstructed at query time, so the index stores the quotient and the residue and nothing else. On real graphs the index is up to seven orders of magnitude smaller than the explicit table it replaces, and it answers point queries in 74 nanoseconds. Its builder outperforms the state of the art by up to 742× in time and 226× in memory on matched cells. One structure answers every cell of the plane.

References

[1] Esra Akbas and Peixiang Zhao. 2017. Truss-based community search: a truss-equivalence based indexing approach. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1298–1309. doi:10.14778/3137628.3137640

[2] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph-based anomaly detection and description: a survey. *Data Mining and Knowledge Discovery* 29, 3 (2015), 626–688.

[3] Vladimir Batagelj and Matjaž Zaveršnik. 2011. Fast algorithms for determining (generalized) core groups in social networks. *Adv. Data Anal. Classif.* 5, 2 (July 2011), 129–145. doi:10.1007/s11634-010-0079-y

[4] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: finding all cliques of an undirected graph. *Commun. ACM* 16, 9 (Sept. 1973), 575–577. doi:10.1145/362342.362367

[5] Paul Burkhardt and Vance Faber. 2020. Bounds and Algorithms for Graph Trusses. *Journal of Graph Algorithms and Applications* 24, 3 (2020), 191–214.

[6] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Software* 38, 1 (2011), 1:1–1:25.

[7] Pierre Dupont, Jérôme Callut, Grégoire Dooms, Jean-Noël Monette, and Yves Deville. 2006. Relevant subgraph extraction from random walks in a graph. https://api.semanticscholar.org/CorpusID:15222702

[8] Andrew Frohmader. 2008. Face Vectors of Flag Complexes. *Israel Journal of Mathematics* 164 (2008), 153–164.

[9] David Gibson, Jon Kleinberg, and Prabhakar Raghavan. 1998. Inferring Web communities from link topology. In *Proceedings of the Ninth ACM Conference on Hypertext and Hypermedia: Links, Objects, Time and Space—Structure in Hypermedia Systems: Links, Objects, Time and Space—Structure in Hypermedia Systems* (Pittsburgh, Pennsylvania, USA) *(HYPERTEXT '98)*. Association for Computing Machinery, New York, NY, USA, 225–234. doi:10.1145/276627.276652

[10] M. Girvan and M. E. J. Newman. 2002. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* 99, 12 (2002), 7821–7826. arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.122653799 doi:10.1073/pnas.122653799

[11] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k-truss community in large and dynamic graphs. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data* (Snowbird, Utah, USA) *(SIGMOD '14)*. Association for Computing Machinery, New York, NY, USA, 1311–1322. doi:10.1145/2588555.2610495

[12] Shweta Jain and C. Seshadhri. 2020. The Power of Pivoting for Exact Clique Counting. In *Proceedings of the 13th International Conference on Web Search and Data Mining* (Houston, TX, USA) *(WSDM '20)*. Association for Computing Machinery, New York, NY, USA, 268–276. doi:10.1145/3336191.3371839

[13] Meng Jiang, Peng Cui, Alex Beutel, Christos Faloutsos, and Shiqiang Yang. 2016. Catching Synchronized Behaviors in Large Networks: A Graph Mining Approach. 10, 4 (2016). doi:10.1145/2746403

[14] Gyula O. H. Katona. 1968. A Theorem of Finite Sets. In *Theory of Graphs*. Akadémiai Kiadó, Budapest, 187–207.

[15] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Washington, D.C.) *(KDD '03)*. Association for Computing Machinery, New York, NY, USA, 137–146. doi:10.1145/956750.956769

[16] Joseph B. Kruskal. 1963. The Number of Simplices in a Complex. In *Mathematical Optimization Techniques*, Richard Bellman (Ed.). University of California Press, Berkeley, 251–278.

[17] Jure Leskovec, Kevin J. Lang, Anirban Dasgupta, and Michael W. Mahoney. 2008. Community Structure in Large Networks: Natural Cluster Sizes and the Absence of Large Well-Defined Clusters. arXiv:0810.1355 [cs.DS] https://arxiv.org/abs/0810.1355

[18] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Mao. 2015. Influential community search in large networks. *Proc. VLDB Endow.* 8, 5 (Jan. 2015), 509–520. doi:10.14778/2735479.2735484

[19] Penghang Liu and A. Erdem Sarıyüce. 2020. Characterizing and Utilizing the Interplay Between Core and Truss Decompositions. In *IEEE International Conference on Big Data*.

[20] László Lovász. 1979. *Combinatorial Problems and Exercises*. North-Holland, Amsterdam.

[21] Fragkiskos D. Malliaros and Michalis Vazirgiannis. 2013. Clustering and community detection in directed networks: A survey. *Physics Reports* 533, 4 (2013), 95–142. doi:10.1016/j.physrep.2013.08.002 Clustering and Community Detection in Directed Networks: A Survey.

[22] Alberto Montresor, Francesco De Pellegrini, and Daniele Miorandi. 2011. Distributed k-Core Decomposition. arXiv:1103.5320 [cs.OH] https://arxiv.org/abs/1103.5320

[23] Francis J O'Reilly, Andrea Graziadei, Christian Forbrig, Rica Bremenkamp, Kristine Charles, Swantje Lenz, Christoph Elfmann, Lutz Fischer, Jörg Stülke, and Juri Rappsilber. 2023. Protein complexes in cells by AI‐assisted structural proteomics. *Molecular Systems Biology* 19, 4 (2023), e11544. arXiv:https://www.embopress.org/doi/pdf/10.15252/msb.202311544 doi:10.15252/msb.202311544

[24] Giulio Rossetti and Rémy Cazabet. 2018. Community Discovery in Dynamic Networks: A Survey. *ACM Comput. Surv.* 51, 2, Article 35 (Feb. 2018), 37 pages. doi:10.1145/3172867

[25] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *AAAI*. https://networkrepository.com

[26] Ryan A. Rossi, Nesreen K. Ahmed, Rong Zhou, and Hoda Eldardiry. 2018. Interactive Visual Graph Mining and Learning, Vol. 9. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3200764

[27] Ahmet Erdem Sariyüce and Ali Pinar. 2016. Fast hierarchy construction for dense subgraphs. *Proc. VLDB Endow.* 10, 3 (Nov. 2016), 97–108. doi:10.14778/3021924.3021927

[28] Ahmet Erdem Sariyüce, C. Seshadhri, and Ali Pinar. 2018. Local algorithms for hierarchical dense subgraph discovery. *Proc. VLDB Endow.* 12, 1 (Sept. 2018), 43–56. doi:10.14778/3275536.3275540

[29] Ahmet Erdem Sariyuce, C. Seshadhri, Ali Pinar, and Umit V. Catalyurek. 2015. Finding the Hierarchy of Dense Subgraphs using Nucleus Decompositions. In *Proceedings of the 24th International Conference on World Wide Web* (Florence, Italy) *(WWW '15)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 927–937. doi:10.1145/2736277.2741640

[30] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Ümit V. Çatalyürek. 2017. Nucleus Decompositions for Identifying Hierarchy of Dense Subgraphs. *ACM Trans. Web* 11, 3, Article 16 (July 2017), 27 pages. doi:10.1145/3057742

[31] Stephen B. Seidman. 1983. Network structure and minimum degree. *Social Networks* 5, 3 (1983), 269–287. doi:10.1016/0378-8733(83)90028-X

[32] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2021. Theoretically and practically efficient parallel nucleus decomposition. *Proc. VLDB Endow.* 15, 3 (Nov. 2021), 583–596. doi:10.14778/3494124.3494140

[33] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2024. Parallel Algorithms for Hierarchical Nucleus Decomposition. *Proc. ACM Manag. Data* 2, 1, Article 32 (March 2024), 27 pages. doi:10.1145/3639287
[34] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The worst-case time complexity for generating all maximal cliques and computational experiments. *Theor. Comput. Sci.* 363, 1 (Oct. 2006), 28–42. doi:10.1016/j.tcs.2006.06.015
[35] Jia Wang and James Cheng. 2012. Truss decomposition in massive networks. *Proc. VLDB Endow.* 5, 9 (May 2012), 812–823. doi:10.14778/2311906.2311909
[36] Wenqian Zhang, Zhengyi Yang, Dong Wen, Yi Ding, Wenjie Zhang, and Xuemin Lin. 2026. Nucleus Decomposition Revisited: An Efficient Counting-Based Approach. *Proc. ACM Manag. Data* 4, 3, Article 215 (May 2026), 26 pages. doi:10.1145/3802092
[37] Wenqian Zhang, Zhengyi Yang, Dong Wen, and Xiaoyang Wang. 2023. Efficient Distributed Core Graph Decomposition. In *2023 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 1023–1031.
[38] Feng Zhao and Anthony K. H. Tung. 2012. Large scale cohesive subgraphs discovery for social network visual analysis. *Proc. VLDB Endow.* 6, 2 (Dec. 2012), 85–96. doi:10.14778/2535568.2448942

A Deferred Constructions

A.1 Building the Boxes

The construction lifts the pivoted clique-tree recursion of [12] from vertices to weighted classes. Form the *class graph*: one node per class, weighted by the class size. Two classes are adjacent when their vertices are pairwise adjacent in G, which by Lemma 4.2 holds exactly when some maximal clique contains both. Compute a degeneracy order of the class graph by repeatedly removing a minimum-degree node. Every clique of G projects to a set of pairwise-adjacent classes, and that set has a unique earliest class in the degeneracy order, its *seed*. For each seed c and each count $j \in [1, |c|]$, the builder opens a box with the fixed entry (c, j). It then runs the canonical pivot recursion of [12] over the neighbors of c that come after it in the order. The recursion fixes some classes at exact counts and releases the others as free, which yields the per-class bounds ℓ and u of Definition 5.1. The bounds are trimmed to the size window $[r_{\min}+1, s_{\max}]$, so one family of boxes serves every witness size of the plane.

Proof of Lemma 5.2. Every clique determines its seed uniquely, so cliques of different seeds lie in different boxes. Within one seed, the canonical branching of the pivot recursion assigns each clique to exactly one leaf, which is the disjointness invariant of the clique tree [12], carried unchanged by the class weights. Each box therefore receives every clique that matches its bounds and no other, and the boxes of one seed cover all cliques seeded there, so the family partitions the cliques of every size in the window. Finally, the recursion only ever adds classes adjacent to everything already on the path, so the classes of one box are pairwise adjacent. □

Per seed, the builder touches only the classes after it in the degeneracy order. Its cost is therefore governed by the degeneracy of the class graph, not by the number of classes. The construction is a negligible share of the build: 1.3 seconds on pkustk13 and 2.5 on web-Google, against plane builds of 217 and 599 seconds.

A.2 Proof of the Shadow Bound

We restate Lemma 6.3: for $s \geq r + 2$ and every r-clique R, $\kappa_{s-1}(R) \geq g_{s-r}(\kappa_s(R))$.

Proof. Let $k = \kappa_s(R)$, realized by the family $\mathcal{S}$ of Lemma 6.1, and let $\mathcal{S}'$ be all $(s-1)$-cliques contained in members of $\mathcal{S}$. Let R' be any r-clique covered by $\mathcal{S}'$, and consider the family $E = \{S \setminus R' : S \in \mathcal{S},\ R' \subseteq S\}$ of $(s-r)$-sets, with $|E| \geq k$. For every $(s-1-r)$-set T in the lower shadow of E, the set $R' \cup T$ is an $(s-1)$-clique inside a member of $\mathcal{S}$, hence a member of $\mathcal{S}'$ containing R'. The Kruskal–Katona theorem in Lovász form [14, 16, 20] bounds the shadow by $g_{s-r}(|E|) \geq g_{s-r}(k)$, so R' lies in at least $g_{s-r}(k)$ members of $\mathcal{S}'$. Since $s - r \geq 2$, dropping one vertex outside R from a member containing R keeps a clique containing R, so $\mathcal{S}'$ covers R. Lemma 6.1 then gives $\kappa_{s-1}(R) \geq g_{s-r}(k)$. □

A sharper shadow theorem is known for flag complexes [8]; the plain bound is the right tool here, since it is already tight at the binomial values that Theorem 6.4 compares.

A.3 Incremental Evaluation of the Filtered Coefficient

Line 10 of Algorithm 2 never re-expands the filtered sum; it applies the following credit rule.

Lemma A.1 (Credit Rule). *Let A_B be the removed compositions of box B and let a be inserted. Call a composition y* newly dominated *if $a \leq y$, no earlier member of A_B is below y, and y respects the bounds of B with $\sum_c y_c = s$. Then for every pattern P hosted by B, $\mathrm{Coef}_s(B, b_P, A_B \cup \{a\}) = \mathrm{Coef}_s(B, b_P, A_B) - \sum_y \prod_c \binom{n_c - b_{P,c}}{y_c - b_{P,c}}$, summed over the newly dominated y with $b_P \leq y$.*

Proof. The index set of Equation (2) for $A_B \cup \{a\}$ is that for A_B minus exactly the newly dominated compositions. Each removed index contributes its product term, which gives the difference. □

The implementation enumerates the newly dominated compositions once per insertion, walking upward from a within the bounds, and credits each co-hosted pattern P with $b_P \leq y$ by the closed-form product. A composition therefore dies exactly once per box over the whole peel. The total update work is proportional to the number of dead-composition and co-hosted-pattern pairs, never to the number of witnesses those compositions stand for.